https://github.com/parnika-vcet/MDM-Practical

```
# Exp 1: EM Spectrum & Wireless Bands (Log-scale chart)
import matplotlib.pyplot as plt

bands = [
    ("ELF/VLF", 3e3, 3e4, "Submarine"),
    ("LF", 3e4, 3e5, "Navigation"),
    ("MF", 3e5, 3e6, "AM Radio"),
    ("HF", 3e6, 3e7, "Shortwave"),
    ("VHF", 3e7, 3e8, "FM, TV"),
    ("UHF", 3e8, 3e9, "GSM, 4G, TV"),
    ("SHF", 3e9, 3e10, "Wi-Fi, 5G, Sat"),
    ("EHF", 3e10, 3e11, "mmWave 5G"),
    ("Infrared", 3e11, 4e14, "Remote"),
    ("Visible", 4e14, 8e14, "Light"),
    ("UV", 8e14, 3e16, "Sterilise"),
    ("X-ray", 3e16, 3e19, "Medical"),
    ("Gamma", 3e19, 1e21, "Nuclear"),
]

fig, ax = plt.subplots(figsize=(13, 4))
ax.set_xscale("log")
ax.set_xlim(1e3, 1e21)

for i, (n, f1, f2, use) in enumerate(bands):
```

```
    ax.barh(0, f2 / f1, left=f1,
            color=plt.cm.rainbow(i / len(bands)),
            edgecolor="k")
    ax.text((f1 * f2) ** 0.5, 0, f"{n}\n{use}",
            ha="center", va="center", fontsize=7, rotation=90)

for f, n, h in [
    (9e8, "GSM 900", 0.85),
    (2.4e9, "Wi-Fi/BT 2.4G", 0.65),
    (5e9, "Wi-Fi 5G", 0.85)
]:
    ax.annotate(n, (f, 0.5), (f, h),
                ha="center", fontsize=7,
                arrowprops=dict(arrowstyle="->"))

ax.set_yticks([])
ax.set_ylim(-0.6, 1.1)
ax.set_xlabel("Frequency (Hz) - log scale")
ax.set_title("Electromagnetic Spectrum & Wireless Bands")

plt.tight_layout()
plt.savefig("exp1_spectrum.png", dpi=150)
plt.show()
```

Exp 1 - EM Spectrum aur Wireless Bands (Case Study)

Frequency and wavelength are connected because when frequency goes up, wavelength goes down. The radio spectrum is divided into bands from LF to EHF to organize wireless services like radio, TV, and 5G. Lower frequencies travel farther and pass through walls easily, while higher frequencies carry more data over a shorter range. A logarithmic scale is used because the frequency range is extremely wide.

Band	Frequency	Use
MF	300 kHz - 3 MHz	AM Radio
VHF	30 - 300 MHz	FM Radio, TV
UHF	300 MHz - 3 GHz	GSM (900/1800), 4G, TV, Bluetooth/Wi-Fi 2.4 GHz
SHF	3 - 30 GHz	Wi-Fi 5 GHz, Satellite, 5G mid-band, Radar
EHF	30 - 300 GHz	5G mmWave

```
# Exp 2: Frequency reuse - drag sliders to change i, j  ->  N = i^2 + i*j + j^2,  Q = D/R = sqrt(3N)
import numpy as np, matplotlib.pyplot as plt
from matplotlib.patches import RegularPolygon
from matplotlib.widgets import Slider

fig, ax = plt.subplots(figsize=(8, 8.5))
plt.subplots_adjust(bottom=0.17)
s_i = Slider(plt.axes([0.2, 0.08, 0.6, 0.03]), "i", 0, 4, valinit=2, valstep=1)
s_j = Slider(plt.axes([0.2, 0.03, 0.6, 0.03]), "j", 0, 4, valinit=1, valstep=1)

def draw(_=None):
    i, j = int(s_i.val), int(s_j.val)
    if i == j == 0: return
    N = i*i + i*j + j*j; Q = np.sqrt(3*N); R = 1.0
```

```
    ax.clear(); cells, keys = [], {}
    for q in range(-12, 13):
        for r in range(-12, 13):
            x, y = np.sqrt(3)*R*(q + r/2), 1.5*R*r
            if np.hypot(x, y) > 10: continue
            k = ((q*(i+j) + r*j) % N, (r*i - q*j) % N)   # same k = same frequency set
            c = keys.setdefault(k, len(keys))
            cells.append((x, y, c))
            ax.add_patch(RegularPolygon((x, y), 6, radius=R, fc=plt.cm.gist_rainbow(c/N), ec="k",
alpha=.8))
            ax.text(x, y, f"F{c+1}", ha="center", va="center", fontsize=7)
    x0, y0, c0 = next(c for c in cells if c[0] == 0 and c[1] == 0)
    near = min((c for c in cells if c[2] == c0 and c[:2] != (x0, y0)), key=lambda c: np.hypot(c[0]-x0,
c[1]-y0))
    ax.plot([x0, near[0]], [y0, near[1]], "k-", lw=3)
    ax.text((x0+near[0])/2, (y0+near[1])/2, f"D={np.hypot(near[0]-x0, near[1]-y0):.2f}R",
ha="center",
            fontweight="bold", bbox=dict(fc="w", ec="k"))
    ax.set_xlim(-10.5, 10.5); ax.set_ylim(-10.5, 10.5); ax.set_aspect("equal"); ax.axis("off")
    ax.set_title(f"i={i}, j={j}   N = {N}   Q = D/R = sqrt(3N) = {Q:.2f}   (same F = same frequencies)")
    fig.canvas.draw_idle()
    print(f"i={i} j={j} N={N}  Q=D/R={Q:.2f}  each cell gets 1/{N} of channels")


s_i.on_changed(draw); s_j.on_changed(draw); draw()
fig.savefig("exp2_reuse.png", dpi=120)   # picture of the starting view (i=2, j=1)
plt.show()
```

Exp 2 - Frequency Reuse (Cellular System)Since the wireless spectrum is limited and users are many, mobile networks reuse the same frequencies in distant cells. Cells are grouped into clusters of size N, and the reuse distance keeps cells with identical frequencies far apart to prevent interference. A larger cluster size reduces interference but gives fewer channels per cell, whereas a smaller cluster increases capacity with higher interference.

i	j	N	Q = D/R	D (from graph)	Channels / cell	Observation
1	1	3	3	3.00 R	1/3	D small, more interference
2	0	4	3.46	3.46 R	1/4	D slightly larger
2	1	7	4.58	4.58 R	1/7	Standard cluster, balanced
2	2	12	6	6.00 R	1/12	D large, less interference
3	1	13	6.24	6.24 R	1/13	D larger still, fewer channels per cell

```
# Exp 7: Multipath Fading
# Sum of many reflected paths (Rayleigh) as receiver moves

import numpy as np
import matplotlib.pyplot as plt

np.random.seed(1)
```

```
f, c = 900e6, 3e8
lam = c / f               # 900 MHz carrier

d = np.linspace(0, 5, 1000)  # Receiver moves 5 m
P = 20                    # Number of paths

amp = np.random.rand(P)
ph = 2 * np.pi * np.random.rand(P)
ang = 2 * np.pi * np.random.rand(P)

# Add all reflected paths
h = sum(
   amp[k] * np.exp(
      1j * (ph[k] + 2 * np.pi * d / lam * np.cos(ang[k]))
   )
   for k in range(P)
)

# Signal level in dB relative to its mean amplitude
rssi = 20 * np.log10(np.abs(h) / np.abs(h).mean())

plt.figure(figsize=(9, 4))
plt.plot(d, rssi)

plt.axhline(
   -10, color="r", ls="--",
   label="deep fade (-10 dB)"
)

plt.xlabel("Distance moved (m)")
plt.ylabel("Signal level (dB)")
```

```
plt.grid(alpha=0.3)
plt.legend()

plt.title("Multipath Fading: 20 reflected paths, 900 MHz")
plt.tight_layout()

plt.savefig("exp7_multipath.png", dpi=150)
plt.show()

print(
    "Max fade depth: %.1f dB, fades below -10 dB: %.1f%% of the time"
    % (rssi.min(), 100 * (rssi < -10).mean())
)
```

Exp 7 - Multipath Effects (Signal Fluctuation)

Wireless signals bounce off buildings and travel along multiple paths to reach the receiver with different delays. When these paths combine in-phase, the signal gets strong, but when they cancel out, it creates a deep fade or sudden drop. Even a tiny movement of the receiver changes the signal significantly due to this fading.

Parameter	Value
Max signal	#ERROR!
Min signal (deepest fade)	-41.4 dB at 3.93 m
Variation (max - min)	48.5 dB
Time below -10 dB	
Deep fades (< -10 dB)	
Wavelength (900 MHz)	

3.50%

5 in 5 m

0.33 m

Where on the graph

highest peak

deepest dip

within only 5 m of movement

below the red dashed line

dips that cross the red line

fade spacing is of the order of the wavelength

Exp 8: Noise & Interference

BER vs SNR for BPSK, with and without interferer

import numpy as np

import matplotlib.pyplot as plt

```
np.random.seed(0)

N = 200000

bits = np.random.randint(0, 2, N)
s = 2 * bits - 1  # BPSK: 0 -> -1, 1 -> +1


def ber(snr_db, sir_db=None):
    # Add AWGN noise
    noise = np.random.randn(N) / np.sqrt(
        2 * 10**(snr_db / 10)
    )

    r = s + noise

    # Add interfering signal, if specified
    if sir_db is not None:
        r += 10**(-sir_db / 20) * np.sign(
            np.random.randn(N)
        )

    # Calculate Bit Error Rate
    return np.mean((r > 0) != bits)


snr = np.arange(0, 11)

plt.figure(figsize=(7, 4.5))
```

```
plt.semilogy(
    snr, [ber(x) + 1e-6 for x in snr],
    "o-", label="Only noise (AWGN)"
)

plt.semilogy(
    snr, [ber(x, 6) + 1e-6 for x in snr],
    "s-", label="Noise + interference (SIR 6 dB)"
)

plt.semilogy(
    snr, [ber(x, 3) + 1e-6 for x in snr],
    "^-", label="Noise + interference (SIR 3 dB)"
)

plt.xlabel("SNR (dB)")
plt.ylabel("Bit Error Rate")
plt.grid(alpha=0.3, which="both")
plt.legend()

plt.title("Effect of Noise & Interference on BER (BPSK)")
plt.tight_layout()

plt.savefig("exp8_noise.png", dpi=150)
plt.show()
```

Exp 8 - Noise aur Interference Analysis

Wireless links face unwanted background noise and interference from other transmitters, measured using SNR and SIR. Bit Error Rate (BER) counts the ratio of incorrectly received bits during transmission. In pure noise, increasing SNR lowers errors quickly, but strong interference creates a flat error floor where extra power does not help much.

Case	Curve on the graph	BER at SNR = 10 dB (approx)
Noise only	Falls steeply	almost 0 (very low)
Noise + interference (SIR 6 dB)	Falls slowly	~1 bit in 150 is wrong
Noise + interference (SIR 3 dB)	Stays high	~1 bit in 20 is wrong

exp10:

```
package com.vcet.calculator;
import android.graphics.Color;
import android.os.Bundle;
import android.view.Gravity;
import android.widget.*;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
TextView tv;
String cur = "";
// number being typed
   double a = 0;          // first number
   String op = "";        // + - × ÷ %
   String[] keys = {"C", "⌫", "%", "÷", "7", "8", "9", "×", "4", "5", "6", "−",
                    "1", "2", "3", "+", "0", ".", "="};

   @Override
   protected void onCreate(Bundle b) {
      super.onCreate(b);
      if (getSupportActionBar() != null) getSupportActionBar().hide();

      LinearLayout root = new LinearLayout(this);         // whole screen (no XML needed)
      root.setOrientation(LinearLayout.VERTICAL);
```

```
    root.setBackgroundColor(0xFF121212);
    root.setFitsSystemWindows(true);
    root.setPadding(24, 24, 24, 24);

    tv = new TextView(this);                        // display
    tv.setText("0"); tv.setTextColor(Color.WHITE); tv.setTextSize(56);
    tv.setGravity(Gravity.BOTTOM | Gravity.END);
    root.addView(tv, new LinearLayout.LayoutParams(-1, 0, 1));

    GridLayout grid = new GridLayout(this);             // 4 x 5 buttons
    grid.setColumnCount(4); grid.setRowCount(5);
    root.addView(grid, new LinearLayout.LayoutParams(-1, 0, 2));

    for (String k : keys) {
      Button btn = new Button(this);
      btn.setText(k); btn.setTextSize(26);
      boolean top = k.matches("C|⌫|%"), orange = "÷×−+=".contains(k);
      btn.setTextColor(top ? Color.BLACK : Color.WHITE);
      btn.setBackgroundTintList(android.content.res.ColorStateList.valueOf(
          orange ? 0xFFFF9F0A : top ? 0xFFA5A5A5 : 0xFF333333));
      int span = k.equals("0") ? 2 : 1;
      GridLayout.LayoutParams p = new GridLayout.LayoutParams(
          GridLayout.spec(GridLayout.UNDEFINED, 1f),
          GridLayout.spec(GridLayout.UNDEFINED, span, (float) span));
      p.width = 0; p.height = 0; p.setMargins(6, 6, 6, 6);
      grid.addView(btn, p);
      btn.setOnClickListener(v -> press(k));
    }
    setContentView(root);
  }
```

```
    void press(String t) {
        if (t.matches("[0-9]")) cur += t;                    // digit
        else if (t.equals(".") && !cur.contains(".")) cur += cur.isEmpty() ? "0." : ".";
        else if (t.equals("C")) { cur = ""; op = ""; a = 0; }
        else if (t.equals("⌫")) { if (!cur.isEmpty()) cur = cur.substring(0, cur.length() - 1); }
        else if (t.equals("=")) {
            if (!op.isEmpty() && !cur.isEmpty()) { cur = fmt(calc(a, Double.parseDouble(cur), op)); op =
"""; }
        } else if ("+−×÷%".contains(t) && !cur.isEmpty()) {     // operator
            if (!op.isEmpty()) cur = fmt(calc(a, Double.parseDouble(cur), op));
            a = Double.parseDouble(cur); op = t; cur = "";
        }
        tv.setText(cur.isEmpty() ? (op.isEmpty() ? "0" : fmt(a) + " " + op) : (op.isEmpty() ? cur : fmt(a)
+ " " + op + " " + cur));
    }

    double calc(double x, double y, String o) {
        switch (o) {
            case "+": return x + y;
            case "−": return x - y;
            case "×": return x * y;
            case "÷": return y == 0 ? Double.NaN : x / y;
            default:  return x % y;
        }
    }

    String fmt(double d) {
        if (Double.isNaN(d)) return "Error";
        return d == (long) d ? String.valueOf((long) d) : String.valueOf(d);
    }
}
```

Exp 10 - Basic Android App: Calculator

Android apps are made of Activities for screen logic, a UI layout, and a Manifest file for app details. In this app, the user interface is built entirely using Java code with a display screen and a grid of buttons. A click listener processes user button presses to perform basic arithmetic and handle errors like division by zero.